

A Real-Time, Robust, and Versatile Visual-SLAM Framework Based on Deep Learning Networks

Xiao Zhang¹, Hongbin Dong², Haoxin Zhang³, Xiaozhou Zhu⁴, Shuaixin Li⁵, and Baosong Deng⁶

Abstract—In this article, we investigate the paradigm of deep learning techniques to enhance the performance of visual-based simultaneous localization and mapping (vSLAM) systems, particularly in challenging environments. By leveraging deep feature extraction and matching methods, we propose a robust, versatile hybrid visual SLAM framework, Rover-SLAM, aimed at improving adaptability in adverse conditions, such as dynamic lighting conditions, areas with weak textures, and camera jitter. Building on excellent learning-based algorithms of recent years, we designed from scratch a novel system that uses the same feature extraction and matching approaches for all SLAM tasks. Our system supports multiple modes, including monocular, stereo, monocular-inertial, and stereo-inertial configurations, offering flexibility to address diverse real-world scenarios. Through comprehensive experiments conducted on publicly available datasets and self-collected data, we demonstrate the superior performance of our Rover-SLAM system compared to the state-of-the-art (SOTA) approaches. We also conducted an in-depth analysis of the integration of visual SLAM with deep learning methods quantitatively to provide insights for future research endeavors in this domain. The experimental results showcase the system's capability of achieving higher localization accuracy and robust tracking performance.

Index Terms—Deep learning, feature matching, hybrid simultaneous localization and mapping (SLAM), robustness, visual inertial navigation.

I. INTRODUCTION

SIMULTANEOUS localization and mapping (SLAM) is a critical technology in the domains of robotics, autonomous driving, and 3-D reconstruction. It concurrently determines the six degree of freedom (6DoF) sensor poses while building a

3-D map of the traversed environment [1]. Vision and inertial sensors are the most commonly employed sensing devices for this purpose, and numerous solutions utilizing these sensors have been extensively studied and explored. After decades of development, the processing architecture of visual (inertial) SLAM has a fundamental framework, including tracking, mapping, and loop closure. In the tracking module, the sensor's pose (position and orientation) is continuously estimated during real-time movement, typically involving feature extraction and matching. Subsequently, In the mapping module, need to utilize the co-view relationship of the frames received from the tracking module to triangulate the new map points. This process aids in constructing and maintaining the sensor's surrounding environment, which is crucial for subsequent pose optimization. The loop closure detection and pose optimization module serve as vital components for reducing cumulative drift and enhancing system robustness in SLAM. In this module, the bag-of-words (BoWs) model is necessary to identify similar scenes and detect visited locations. This allows for the identification and correction of trajectory drift. Over the past decades, numerous brilliant visual-based SLAM (vSLAM) solutions employing classical computer vision methods have emerged, including ORB-SLAM [2], DVO [3], and MSCKF [4], driving significant evolution in this domain. While many of the foundational issues have been addressed, recent research has focused on enhancing the robustness and adaptability of SLAM under extreme conditions [5]. In challenging environments, such as those with dynamic lighting and areas with weak textures, traditional feature extraction algorithms, which primarily focus on local information in images and often overlook structural and semantic details, substantially limit the effectiveness of pose estimation. As a result, existing SLAM systems may struggle to obtain accurate and stable features in these scenarios, leading to unstable or ineffective tracking.

The rapid advancement of deep learning has revolutionized the field of computer vision, enabling models to assimilate complex scene structures and semantic information through extensive data training, which has made environment perception more intelligent. The integration of deep learning with SLAM follows two routes. The first involves end-to-end algorithms based on deep learning, such as NICE-SLAM [6] and DVI-SLAM [7]. These methods often incur high-computational costs, long optimization times, and face challenges in real-time tracking, and the lack of detailed geometric correspondences can lead to suboptimal positioning accuracy. Furthermore, these methods may excel only in

Received 7 August 2024; revised 18 November 2024; accepted 27 November 2024. Date of publication 13 January 2025; date of current version 4 March 2025. This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 42201501. The Associate Editor coordinating the review process was Dr. Antonio Moschitta. (Corresponding authors: Shuaixin Li; Baosong Deng.)

Xiao Zhang is with the College of Computer Science and Technology, National Engineering Laboratory for E-Government Modeling and Simulation, Harbin Engineering University, Harbin 150009, China, and also with the Defense Innovation Institute, Chinese Academy of Military Science, Beijing 100850, China.

Hongbin Dong is with the College of Computer Science and Technology, National Engineering Laboratory for E-Government Modeling and Simulation, Harbin Engineering University, Harbin 150009, China.

Haoxin Zhang is with the Defense Innovation Institute, Chinese Academy of Military Science, Beijing 100850, China, and also with the College of Systems Science and Engineering, Sun Yat-sen University, Guangzhou 510275, China.

Xiaozhou Zhu, Shuaixin Li, and Baosong Deng are with the Defense Innovation Institute, Chinese Academy of Military Science, Beijing 100850, China, and also with the Intelligent Game and Decision Laboratory, Beijing 100166, China (e-mail: lxx_navigation@sina.com; dbs@nudt.edu.cn).

Digital Object Identifier 10.1109/TIM.2025.3527618

scenarios that closely resemble their training datasets. When confronted with unfamiliar environments, their estimated trajectories tend to be inaccurate or even invalid. The second route, known as hybrid SLAM, enhances specific SLAM modules using deep learning. This approach balances traditional geometric methods and deep learning techniques, leveraging their strengths to balance geometric constraints and semantic understanding. Although there is ongoing research in this area, effectively integrating deep learning techniques remains a worthwhile direction for further study. Despite their advantages, existing hybrid SLAM systems have limitations. For instance, DX-Net [8] simply replaces ORB features with deep features but relies on traditional tracking methods, leading to incoherence in the deep feature information. Similarly, SP-Loop [9] integrates deep learning features only into the loop closure module, retaining traditional feature extraction methods elsewhere. Consequently, there is a need for a versatile hybrid SLAM method that effectively integrates deep learning technology to address complex environmental challenges comprehensively.

In this work, we propose Rover-SLAM, a real-time, robust, and versatile visual-SLAM framework. We incorporate a state-of-the-art (SOTA) learning-based feature extraction module, SuperPoint [10], as the sole representation form throughout the system. Additionally, in complex environments, traditional feature-matching methods often exhibit instability, leading to degraded tracking and mapping quality. Recent advances in learning-based feature matching methods [11] demonstrate superior matching performance when encountering significant changes in viewing angles or challenging scenes, as it can leverage prior scene information and structural details to enhance matching effectiveness. Consequently, we replace traditional feature-matching methods with the learning-based approach LightGlue [12], which enhances matching performance under significant changes in viewing angles. We preprocess the learning-based feature descriptors to align with the training of the corresponding visual vocabulary and design a strategic feature selection mechanism to maintain accuracy and efficiency. Finally, we conduct a series of experiments to demonstrate that the proposed SLAM framework improves both trajectory estimation accuracy and tracking robustness in various challenging scenarios (see Fig. 1). Our contributions to this work include the following.

- 1) We propose a real-time, learning-based vSLAM framework designed to enhance adaptability in challenging environments. By integrating deep learning modules and novel combination schemes throughout the SLAM system, covering tracking, local mapping, and loop closure, we significantly improve both the robustness and accuracy of the SLAM system while maintaining real-time performance.
- 2) We developed the first versatile SLAM system that incorporates deep feature extraction and matching approaches. The proposed system supports various sensor configurations, including monocular, stereo, monocular-inertial, and stereo-inertial, catering to diverse application needs.

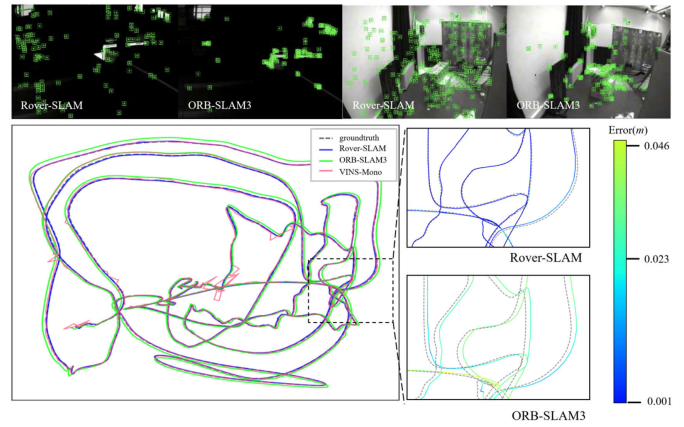


Fig. 1. Rover-SLAM versus other SOTA methods. Comparison of map point tracking performance between the proposed Rover-SLAM and ORB-SLAM3 in a challenging environment. The first and third subfigures show the results of Rover-SLAM, while the second and fourth subfigures show the results of ORB-SLAM3 (top). Comparison of the trajectories obtained using the ORB-SLAM3, VINS-Mono, and Rover-SLAM, in a EuRoC V203 sequence, which is the most challenging sequence for shaking and dynamic lighting reason (bottom left). Map details are color-coded with the amount of error, i.e., green corresponds to higher error levels, and blue to lower ones (bottom right).

- 3) We conducted extensive experiments to demonstrate the effectiveness and robustness of our system. Results from both public datasets and self-collected datasets indicate its superiority over other SOTA SLAM systems. The system is fully implemented in C++ and ONNX, ensuring real-time operation. For the benefit of the research community, the source code is available at <https://github.com/zzzx111/SLslam>.

II. RELATED WORK

A. Traditional Visual SLAM Works

The development of SLAM has been significantly influenced by traditional geometric methodologies, resulting in numerous classic and successful approaches. In particular, direct sparse odometry (DSO) [13] exemplifies direct SLAM by sampling pixels with intensity gradients in images and integrating a photometric error model and all model parameters into a joint optimization function. ORB-SLAM [2], a representative sparse feature-based SLAM system, utilizes ORB features to describe scenes. It employs three parallel threads for tracking, local mapping, and loop closure, allowing trajectory estimation and optimization. At the same time, it also leverages the image pyramid to enhance the scale invariance and robustness of feature extraction and matching. With the growing trend of assisting the vision system with a low-cost inertial measurement unit (IMU), the visual-inertial navigation system (VINS) has gained prominence. VINS-Mono [14] integrates pre-integrated IMU measurements and feature observations to achieve high-precision visual-inertial odometry (VIO), forming a complete monocular VIO-SLAM system with loop closure detection and graph optimization. Additionally, it employs the Kanade–Lucas–Tomasi (KLT) pyramid optical flow method to track images, enhancing the stability of tracking in challenging scenarios. FP-SLAM [15]

argues that point features lack robustness in low-texture and illumination-varying environments. Therefore, it integrates line features with structural information to compensate for the weaknesses of point features, and optimizes the point-line feature relationship while improving the LSD algorithm to design a dynamically fused point-line VINS, enhancing localization accuracy and adaptability in complex scenes. ORB-SLAM3 [16] extends the capabilities of ORB-SLAM by incorporating inertial data to improve performance. Liu et al. [17] proposed a lightweight SLAM method that combines cloud computing and edge computing, enabling real-time collaborative localization and mapping. However, its drawback is the reliance on network bandwidth, which may lead to poor performance in high-latency or low-bandwidth environments.

A common issue with these traditional image processing-based SLAM systems is their susceptibility to challenging environments, which can lead to inaccurate pose estimation or even lost tracking.

B. Learning-Based Visual SLAM Works

To address the aforementioned issue, researchers are increasingly incorporating deep learning methodologies into SLAM systems to enhance robustness in challenging scenarios. One class of approaches involves training end-to-end models for visual SLAM [18]. For instance, UnDeepVO [19] is notable as the first end-to-end visual odometry (VO) framework based on neural networks, leveraging stereo training to recover absolute scale by utilizing spatial and temporal geometric constraints. D3VO [18] addresses the challenge of image depth estimation in occluded and dynamic environments by incorporating a Bayesian filtering method, which quantifies uncertainty for each pixel and thereby improves accuracy and performance in pose estimation. DROID-SLAM [20] introduces a micro-loop optimization heuristic to enhance the network performance and generalization capabilities. KN-SLAM [21] integrates the local feature points method into the neural implicit map-based SLAM system to enhance camera pose estimation accuracy, and incorporates feature reprojection loss and photometric loss into the system to ensure higher accuracy in trajectory estimation. DPVO [22] utilizes a feature-based patch representation to encode local scene information. It employs a novel recurrent network structure to incorporate temporal information for tracking these image patches. While the aforementioned end-to-end methods offer adaptability across various application scenarios, they typically require extensive datasets and significant computing resources for training. Additionally, although they are capable of acquiring semantic and structural information about the scene, they still have shortcomings in real-time performance. Furthermore, when the test scene differs significantly from the training scene, there may be a certain degree of degradation in localization accuracy.

Another approach is to integrate learning-based methods into specific SLAM systems, known as hybrid SLAM [23]. Learned invariant feature transform (LIFT)-SLAM [24] incorporates the LIFT network for feature extraction within the ORB-SLAM system's backend. Similarly, GCNv2SLAM

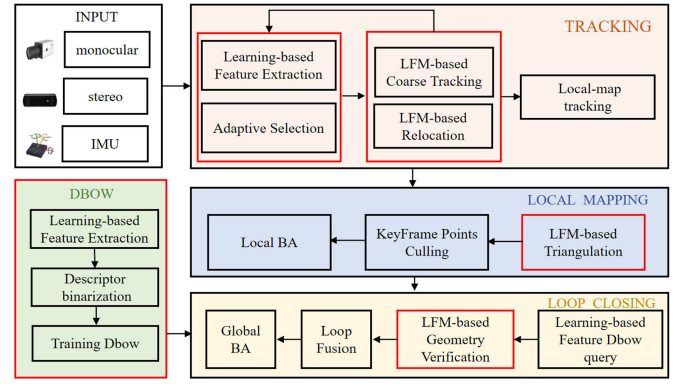


Fig. 2. Framework of Rover-SLAM. The system consists of tracking, local mapping, and loop-closing modules. LFM is short for “learnable feature matching.” The improvements proposed in this framework are highlighted with red rectangular boxes.

[23] employs a geometric correspondence network (GCN) to extract feature points and descriptors of the scene. DX-SLAM [8] replaces ORB features in ORB-SLAM with learning-based features and uses the OpenVINO toolkit for model deployment. However, these approaches still rely on traditional methods for feature matching during tracking. Due to the poor performance of traditional feature-matching methods when faced with challenges such as weak textures, photometric changes, and large baseline transformations, the robustness of the aforementioned works is inadequate. CD-SLAM [25] adopts a single-stage YOLOv5 [26] network. By parallelly executing YOLOv5 object detection and tracking threads, it utilizes semantic information as prior knowledge for pose estimation, thereby optimizing localization accuracy. ST-VIO [27] first uses a direct method to track keyframes of images and employs a Siamese network for image segmentation and tracking. This enables mid-term tracking, allowing the system to estimate pose through overall feature matching in large-scale movements. Fu and Yao [28] integrated a Transformer-based feature extraction network into a cloud computing platform to enhance the robustness of the SLAM system. However, this approach has high requirements for hardware and network, which limits its real-time performance. AirVO [29] uses a learning-based feature points extraction and matching method, optimizing the pose by jointly minimizing point and line feature reprojection errors to improve estimation accuracy. Nonetheless, AirVO is not a complete SLAM system and is limited to stereo accessible environments.

III. METHOD

A. System Overview

The pipeline of the proposed Rover-SLAM builds upon the standard parallel visual SLAM paradigm [30] and follows the architectural design of ORB-SLAM3 (see Fig. 2). The system is divided into three modules, i.e., tracking, local mapping, and loop closure, and supports four configurations, i.e., monocular, monocular-inertial, stereo, and stereo-inertial. In our implementation, the ONNX Runtime deep learning deployment framework is employed, incorporating the SOTA learning-based methods, SuperPoint and LightGlue. Notably,

our framework is flexible and not restricted to specific deep learning models, allowing for replacements such as XFeat [31] or SuperGlue [32].

In the SLAM system, the Tracking thread is responsible for real-time tracking of the device's pose using sensor data. The local mapping thread needs to construct and update the local map in real time while the device is moving, and it jointly optimizes the pose using the local map to enhance localization accuracy. Loop closure identifies when the device returns to known locations, effectively correcting accumulated localization errors and improving the consistency of the global map. These three threads work together, building short-term, mid-term, and long-term associations, enabling the algorithm to achieve stable localization and mapping in complex environments.

B. Learning-Based Feature Extraction and Matching

1) *Learning-Based Feature Extraction*: Traditional feature corner extraction methods typically rely on pixel intensity and neighborhood information. Compared to deep learning-based feature extraction techniques, these traditional approaches exhibit poorer invariance to scale, illumination, and rotation, making them less adaptable to complex environments. Consequently, the proposed framework employs the effective SuperPoint feature extraction network. This network comprises a shared encoder, a feature detection decoder, and a descriptor decoder, outputting a probability tensor of features and a descriptor tensor, respectively. The structure of the feature extraction module is illustrated in Fig. 3. Each grayscale image $\mathbf{I} \in \mathbb{R}^{H \times W}$ is fed into the network, producing the score tensor $\mathbf{S} \in \mathbb{R}^{H \times W}$ and the descriptor tensor $\mathbf{D} \in \mathbb{R}^{1 \times L \times (W/8) \times (H/8)}$. Note that, an input image that is too small may hinder feature extraction, deteriorating tracking performance, while excessively large images could result in high computational demands and memory usage. Features with score tensors and descriptors are filtered using a confidence threshold T_f . This process is as shown in the following equation, where $i \in \{1, \dots, H\}$, $j \in \{1, \dots, W\}$, $\mathcal{F}$ represents the feature point set, and $\mathbf{F}_{ij}$ represents the feature point at (i, j) :

$$\mathcal{F} = \{\mathbf{F}_{i,j} | (i, j), \mathbf{S}_{i,j} > T_f\}. \quad (1)$$

2) *Adaptive Confidence Threshold*: However, a fixed confidence threshold T_f can result in most extracted features being filtered out as the confidence of each feature point diminishes in challenging scenes. To address this issue, we introduce a self-adaptive strategy to dynamically adjust T_f according to the scene, enabling more robust feature extraction in challenging scenarios. The adaptive threshold mechanism considers two factors.

- 1) The first is intra-feature relationship which refers to the distribution of feature points across all positions within a frame. We calculate the expectation E and variance σ^2 of the confidence distribution of the feature points within each frame. Assuming the confidence distribution follows a Gaussian distribution, the number of feature points within the range $[E + (\sqrt{\sigma^2}/2), +\infty)$ should

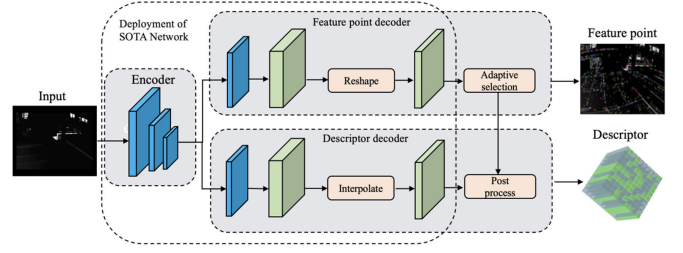


Fig. 3. Overview of feature extraction. Using SuperPoint as the feature extraction network, which includes an encoder, a feature decoder, and a descriptor decoder. Subsequently, an adaptive filter is applied to sort out feature points.

remain constant, ensuring consistent feature extraction and avoiding situations where fixed thresholds may fail to extract sufficient feature points.

- 2) The second is the interframe feature relationship which assesses the match quality between the current frame and adjacent frames. Since the interval between each frame is small, the matching quality of the previous frame can roughly reflect the current frame's matching quality. Let m denote the number of matched features between the current frame and adjacent frames. A decrease in matches signals a weak texture area or significant motion, indicating the need for extracting more feature points for subsequent tracking. Conversely, a strong match suggests a rich texture area, allowing for reduced feature extraction and lower computational costs. The adaptive threshold calculation formula is given in the following equation, where μ_1 and μ_2 are hyperparameters:

$$T_f = E + \frac{\sqrt{\sigma^2}}{2} + \mu_1 \frac{1}{(1 + \exp(-\mu_2 m))}. \quad (2)$$

3) *Feature Matching Network*: The LightGlue model comprises multiple identical layers processing two feature sets collectively. Each layer contains self-attention and cross-attention units to update point representations. A classifier in each layer determines inference halting, avoiding unnecessary computations. Finally, a lightweight head computes partial match scores. The network depth dynamically adjusts based on input image complexity. If image pairs are easily matched, early termination is possible due to high token confidence. Therefore, LightGlue has a shorter runtime and lower memory consumption, making it suitable for integration into tasks that require real-time performance.

Given two sets of feature points extracted from images $\mathbf{I}_A$ and $\mathbf{I}_B$, where M and N feature points are extracted, respectively, we denote the features of image $\mathbf{I}_A$ as $\mathcal{A} = \{f_i^A\}$, $0 \leq i \leq M$ and those of image $\mathbf{I}_B$ as $\mathcal{B} = \{f_j^B\}$, $0 \leq j \leq N$. Each feature point descriptor is normalized to fall within the range of 0–1. After normalization, the feature points are fed into the learning-based feature matching network, which outputs the soft partial assignment matrix $\mathbf{P} \in [0, 1]^{M \times N}$. The element $p_{ij} \in \mathbf{P}$ represents the likelihood that the i th feature point of $\mathbf{I}_A$ matches the j th feature point of $\mathbf{I}_B$. LightGlue is utilized for its capability to dynamically

adjust the network depth and terminate early based on the input image's complexity. This feature significantly reduces the running time and memory consumption, making it highly suitable for SLAM tasks that require real-time performance.

C. Deep BoW Descriptor

The BoW approach used in SLAM system is a visual vocabulary-based method, adapted from the BoWs concept in natural language processing. It starts with offline dictionary training, clustering feature descriptors from the training image set into k sets using K -means, forming the first level of the dictionary tree. This process is recursively applied within each set, resulting in a final dictionary tree with depth L and branch number k . Each leaf node in this visual vocabulary tree is called a word.

During algorithm execution, BoW vectors and feature vectors are generated online from all features of the query image. Mainstream SLAM frameworks prefer manually set binary descriptors for their small memory requirements. To further enhance efficiency, we use a Gaussian distribution with an expected value of 0 and a standard deviation of 0.07 for the SuperPoint descriptor in our system. Consequently, the 256-D floating-point SuperPoint descriptor d can be binary encoded to improve the indexing speed of visual place recognition. The binary encoding formula is given in (3), where $\mathbf{d}_i$ is the i th element of descriptor $\mathbf{d}$, $0 \leq i < 256$. The binary encoding of deep feature descriptors improves retrieval efficiency in three ways: by compressing storage space, accelerating the training process, and enhancing online queries [33]

$$\mathbf{d}_i = \begin{cases} 1, & \text{if } \mathbf{d}_i \geq 0 \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

D. System Structure Description

1) *Tracking Based on Learnable Feature Matching*: For each incoming image, the system initially processes it through the learning-based feature extraction network. The system then utilizes two frames for initialization and conducts coarse tracking on each subsequent frame. In coarse tracking, a feature-matching algorithm that relies on the similarity of feature descriptors between the query and target images is used. To speed up and reduce costs, the search area in the target image is restricted based on estimated positions predicted by camera motion. With short intervals between frames, VO assumes uniform motion, while VIO uses IMU integration. Obviously, rapid acceleration or rotation can reduce this strategy's reliability due to posing prediction deviations. To address this issue, the learning-based method offers a viable solution. By directly comparing features between current and previous frames in real-time, the matching network enhances robustness to aggressive motion, ensuring more reliable feature matching despite sudden changes in camera dynamics. In our implementation, LightGlue, equipped with a graph neural network (GNN) submodule capable of cross-view matching, is employed.

If continuous tracking fails, feature matching resorts to the latest keyframe, requiring descriptor-based matching.

Algorithm 1 Tracking Based on Learnable Feature Matching

Input: $F = \text{frame}$
Output: $KF = \text{keyframe}$, $\mathbf{T}^r = \text{refined pose}$

```

1 for  $F = \text{each incoming frame}$  do
2    $FP \leftarrow \text{featureExtraction}(F)$   $\{FP = \text{feature points}\}$ 
3   if !initialized then
4      $\mathbf{T}, M \leftarrow \text{LFMBasedInitialization}(F)$   $\{\mathbf{T} = \text{pose},$ 
5        $M = \text{Map}\}$ 
6   end
7    $\mathbf{T}^c \leftarrow \text{LFMBasedCoarseTracking}(F)$ 
8    $\{\mathbf{T}^c = \text{coarse pose}\}$ 
9   if trackingSuccess then
10     $\mathbf{T}^r \leftarrow \text{localMapTracking}(\mathbf{T}^c, F)$ 
11    if trackingSuccess then
12       $KF \leftarrow \text{creatingKeyframe}(F)$ 
13       $\{KF = \text{keyframe}\}$ 
14    end
15    else
16       $\mathbf{T}^r \leftarrow \text{LFMBasedRelocation}(F)$ 
17    end
18  end
19 else
20    $\mathbf{T}^c \leftarrow \text{LFMBasedReferenceTracking}(F)$ 
21   if trackingSuccess then
22      $KF \leftarrow \text{creatingKeyframe}(F)$ 
23   end
24   else
25      $\mathbf{T}^r \leftarrow \text{LFMBasedRelocation}(F)$ 
26   end
27 end

```

Consequently, feature descriptor-based matching becomes necessary. For example, in ORB-SLAM and VINS-Mono, the BoW algorithm expedites matching between current and reference keyframes for re-localization. However, the BoW approach converts spatial information into statistical information based on visual vocabulary, potentially losing accurate spatial relationships between feature points. To overcome these challenges, a learning-based matching approach is much more suitable. This transition has significantly increased the likelihood of successful tracking under large-scale transformations, thereby improving the accuracy and robustness of the tracking process. The pseudocode for the Tracking thread is shown in Algorithm 1. Notably, a learnable feature-matching network is employed for feature matching in coarse tracking, initialization, reference frame tracking, and relocalization. LFM is short for “learnable feature matching.” This ensures accurate and robust matching relationships, thereby enhancing tracking effectiveness.

2) *Local Mapping Based on Learnable Feature Matching*: The primary role of the local mapping thread is the real-time construction of a local map, incorporating map points and keyframes. It utilizes the local map to execute

Algorithm 2 Local Mapping Based on Learnable Feature Matching

Input: KF = keyframe
Output: LM^u = updated local map, T^u = updated pose

```

1 for each incoming keyframe  $KF$  do
2    $LM \leftarrow \text{processNewKeyFrame}(KF)$  {  $LM$  = local map }
3    $LM^c \leftarrow \text{mappointCulling}(KF, LM)$  {  $LM^c$  = culled local map }
4    $LM^u \leftarrow \text{LFMBasedMappointCreation}(KF, LM^c)$ 
5   if !ImuInitialized then
6     initializeImu( $KF, LM^u$ )
7   end
8    $LM^u, T^u \leftarrow \text{localBundleAdjust}(KF, LM^u)$ 
9    $LM^u \leftarrow \text{keyframeCulling}(KF, LM^u)$ 
10 end
  
```

bundle adjustment (BA) optimization, thereby reducing tracking errors and enhancing consistency.

First, keyframes need to be processed, and map points are filtered based on co-visibility relationships. Then, new map points are triangulated using the current keyframe and its neighboring keyframes. To obtain more accurate map points, it is essential to match with keyframes that exhibit larger parallax. By employing learning-based matching in triangulating the matched points, more comprehensive and higher quality map points can be obtained. This enhances the local mapping capability by establishing additional connections between keyframes and constructing a more robust essential graph. Therefore, local BA can be utilized for better joint optimization of the poses of co-visible keyframes and map points. Redundant map points and keyframes are then identified and removed. In inertial mode, IMU initialization is conducted within this thread. The pseudocode for the tracking thread is shown in Algorithm 2.

3) Loop Closure Based on Learnable Feature Matching:

Loop closure in SLAM typically involves three key stages: seeking initial loop closure candidates, verifying loop closure keyframe, and performing correction by global BA. First, initial loop closure candidates are identified using the previously trained BoW model. Keyframes that share common words with the current frame K_A , but are not co-visible with it, are identified. The collective score of the co-visible keyframes associated with these candidate keyframes is computed. From the top N groups with the highest scores, the keyframe with the highest score is selected, denoted as K_B . Similar to ORB-SLAM3, when a co-visible area is detected, if it occurs between the current frame and the active map, a loop closure is performed. If the co-visible area occurs between the current frame and the inactive map, a map merging operation is executed.

Next, the relative pose transformation T_{AB} from K_B to the current keyframe K_A must be determined. Unlike ORB-SLAM3, we use a learning-based feature matching method instead of a DBOWs-based feature matching approach to match the current keyframe with the candidate keyframe

K_B and its co-visible keyframe K_C . It is worth noting that the learning-based matching algorithm significantly enhances matching accuracy, yielding high-quality map point correspondences. This high-quality correspondence ensures that a more accurate initial relative pose T_{AB} can be obtained using the Sim(3) transformation in subsequent stages. Finally, loop closure fusion and global BA are performed to optimize the overall pose.

IV. EXPERIMENT

A. Implementation Details

In this section, we evaluate the performance of Rover-SLAM with various sensor configurations through both qualitative and quantitative analyses. The assessments cover aspects such as pose estimation accuracy, running time, and robustness. All experiments were conducted on a computer equipped with an Intel Core i7-11700 CPU, 32 GB of RAM, an NVIDIA RTX 3080 GPU, and running Ubuntu 18.04. To ensure the flexibility, scalability, and real-time performance of Rover-SLAM, we deployed the deep learning models using the ONNX Runtime framework. We directly take the pretrained SuperPoint and LightGlue models without any fine-tuning training.

We employ the root mean squared error (RMSE) of the absolute trajectory error (ATE) to evaluate the accuracy of pose estimation. ATE assesses the disparity between the camera pose trajectories output by the SLAM system and the ground-truth camera pose trajectories. RMSE quantifies these errors by computing the square root of the average sum of squared errors over all frames. A smaller RMSE indicates better consistency between the SLAM system's output trajectory and the ground truth trajectory. The formula for ATE is shown in (4). Where N is the total number of frames in the trajectory. T_{est}^i is the position vector of the estimated trajectory at the i th frame. T_{gt}^i is the position vector of the ground-truth trajectory at the i th frame. $\|\cdot\|$ represents the Euclidean distance

$$\text{RMSE}_{\text{ATE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|T_{\text{gt}}^i - T_{\text{est}}^i\|_2^2}. \quad (4)$$

B. Datasets

To evaluate the localization accuracy of Rover-SLAM, we conduct comprehensive assessments on two used public datasets: EuRoC [34] and TUM-VI [35].

1) *EuRoC*: The EuRoC dataset encompasses recordings from office and factory environments, categorized into three difficulty levels: easy (MH01/2, V101, V201), medium (MH03/4, V102, V202), and difficult (MH05, V103, V203). The difficult level is characterized by variable lighting conditions, severe shaking, and dark environments, providing a range of challenging scenarios. This dataset images captured by two global-shutter, monochrome cameras at a rate of 20 Hz, alongside angular rates and specific force measurements logged from the integrated IMU at 200 Hz. This comprehensive setup allows for testing various sensor configurations, including monocular-inertial, stereo-inertial, stereo, and monocular modes.

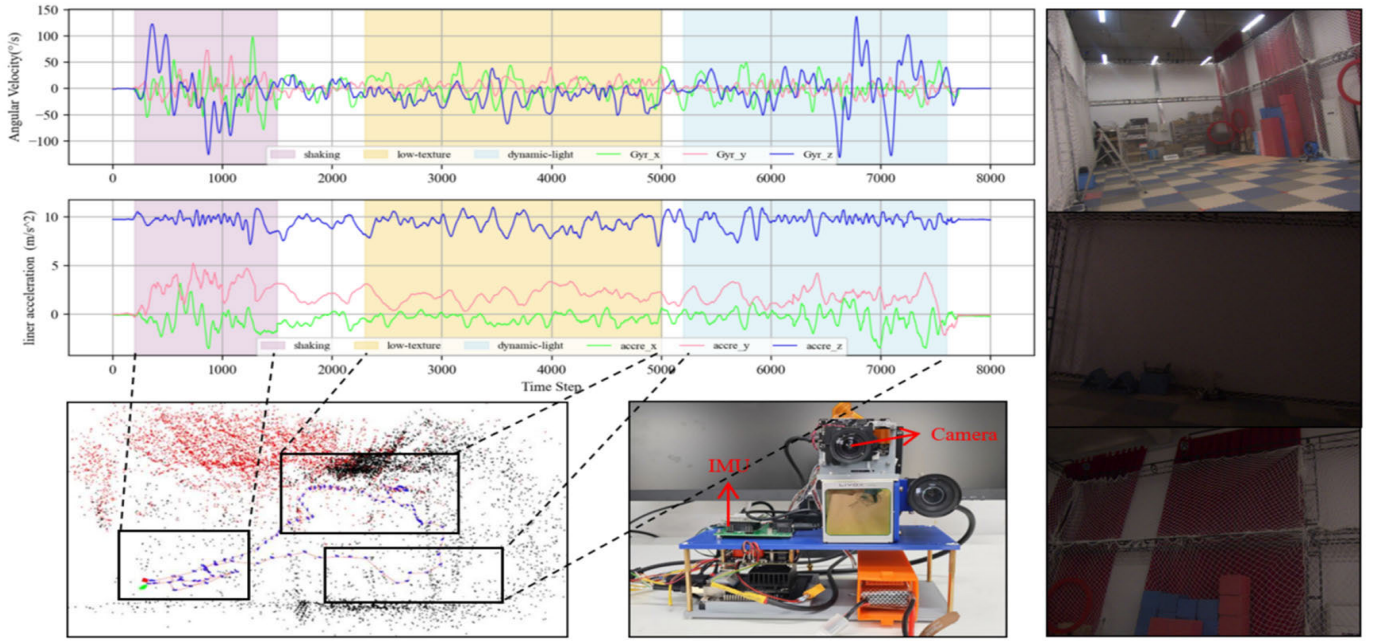


Fig. 4. Self-collected dataset test. Self-collected sequence characteristics in various challenge environments (top left). The shaded area in different colors, respectively, represents different phases of shaking, low texture, and dynamic light. Estimated trajectory of Rover-SLAM on this sequence (bottom left). Schematic of data collection equipment (bottom middle). Schematic of data collection environment (bottom right).

2) *TUM-VI*: The TUM-VI Benchmark is a novel dataset designed for evaluating VIO methods across various scenes with multiple sequences. It provides camera images with a resolution of 1024×1024 at 20 Hz. The IMU's Measurement at a frequency of 200 Hz. For trajectory evaluation, accurate pose ground truth is provided at the beginning and end of sequences from a motion capture system. We use it to test mainstream monocular-inertial sensor setups.

3) *Self-Collected Dataset*: We collected data using our own equipment in various challenging environments, including changes in illumination, low texture, and shaking scenarios. Our handheld device for data sampling consists of a HIKVISION monocular camera with resolution of 1440×1080 , frequency of 20 Hz, and an Xsense IMU with frequency of 100 Hz. Our handheld device is shown in Fig. 4 (bottom middle), and the sensors used are labeled in this figure. We use rosbag to record images and IMU data. To conveniently simulate challenging environments, we chose to collect the dataset indoors. We simulated illumination variation by rapidly switching the lights on and off, created low-texture scenarios by photographing less textured walls, and modeled shaking conditions by quickly moving the device. The specifics of our sequence's details are shown in Fig. 4 (top left), where we use different colors to indicate the challenging environments.

C. Accuracy Comparison

Since the EuRoC dataset provides both binocular images and inertial data, it allows us to test different sensor configurations, including monocular-inertial, stereo-inertial, and monocular modes. Notably, conducted extra in-depth tests on the monocular-inertial mode using the TUM-VI dataset, as it is the most commonly used sensor type in robotic applications.

Additionally, we selected the open-source versatile vSLAM solution, ORB-SLAM3, for a comprehensive comparison.

1) *Monocular-Inertial Tests*: In the **EuRoC dataset tests**, we selected several SOTA SLAM methods as contrast objects, including traditional visual-inertial SLAM methods like DFP-SLAM [15], ST-VIO [27], VINS-Mono [14], ORB-SLAM3 [16], the hybrid SLAM algorithm SP-Loop [9], and the SOTA end-to-end deep learning SLAM system DVI-SLAM [7]. We compared the trajectories from both quantitative and qualitative perspectives. The quantitative results of the ATEs in terms of RMSE are presented in Table I. Rover-SLAM exhibited significant advantages over other approaches, leading in 8 out of the 11 sequences, with an average superiority of over 0.08 m compared to ORB-SLAM3. It also outperformed other traditional algorithms by a large margin. In comparison to other deep learning-based SLAM systems, Rover-SLAM exhibited notable improvements in state estimation accuracy. This can be attributed to the extraction and matching of more stable and distinctive feature points, resulting in more complete and accurate estimations. The visualizations of the trajectory and error in each direction are shown in Fig. 5. When ORB-SLAM3 encounters challenging areas, the tracking becomes unstable and the tracking error increases. In contrast, Rover-SLAM maintains more stable and accurate positioning in these challenging scenarios.

In the **TUM-VI dataset tests**, we conducted experiments using ORB-SLAM3 to comprehensively illustrate the advantages of the proposed Rover-SLAM. This dataset was captured using handheld devices and evaluated with VIO. Ground truth poses are provided only at the beginning and end of the trajectory, representing a small portion of the entire path. Therefore, we followed the methodology of ORB-SLAM3 to evaluate the accumulated drift over the entire trajectory. The results for

TABLE I
RMSE[M] OF ATE COMPARISON WITH SOTA VINS METHODS ON EUROC DATASET. × DENOTES FAILURE OF THE CORRESPONDING APPROACH FOR THIS SEQUENCE. THE BEST RESULT IS HIGHLIGHTED IN BOLD

Dataset	DFP-SLAM	ST-VIO	OKVIS	VINS-Mono	ORB-SLAM3	DVI-SLAM	SP-Loop	IMPS	Ours
V101	0.135	0.095	0.090	0.146	0.091	0.074	0.042	0.090	0.034
V102	0.287	0.134	0.122	0.311	0.063	0.114	0.034	0.061	0.019
V103	0.381	0.145	0.196	0.329	0.066	0.083	0.082	0.061	0.031
V201	0.122	0.053	0.168	0.124	0.076	0.091	0.038	0.074	0.024
V202	0.259	0.104	0.182	0.277	0.058	0.045	0.054	0.056	0.014
V203	0.341	0.280	0.305	0.323	0.063	0.072	0.100	0.065	0.018
MH01	0.168	0.219	0.292	0.177	0.044	0.063	0.070	0.039	0.048
MH02	0.173	0.156	0.361	0.183	0.083	0.066	0.044	0.058	0.049
MH03	0.405	0.175	0.267	0.404	0.044	0.101	0.068	0.040	0.039
MH04	0.396	0.286	0.366	0.394	0.082	0.187	0.100	0.054	0.052
MH05	0.384	0.352	0.396	0.382	0.064	0.163	0.09	0.058	0.048

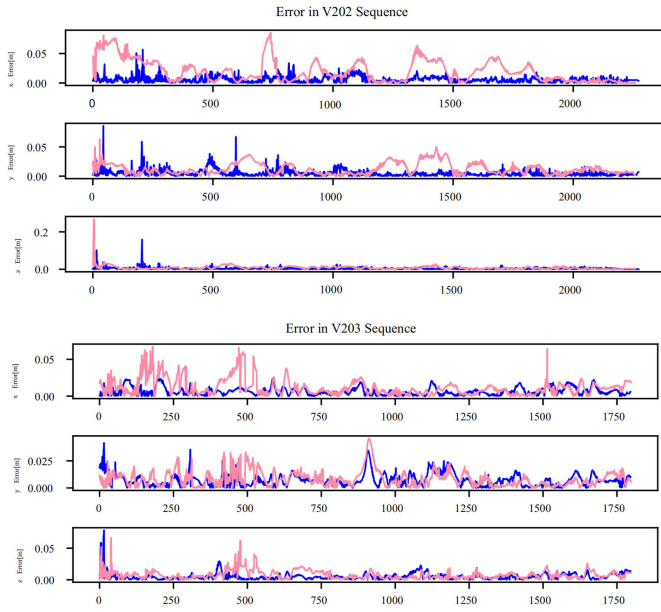


Fig. 5. Comparison of tracking error in EuRoC's V202 and V203 sequences between Rover-SLAM and ORB-SLAM3.

TABLE II
RMSE[M] OF ATE WITH SOTA VINS METHODS ON TUM-VI

Dataset	VINS-Mono	ORB-SLAM3	IMPS	Ours	length
corridor1	0.63	0.23	0.19	0.086	305
corridor2	0.95	0.04	0.04	0.062	322
corridor3	1.56	0.40	0.35	0.039	300
corridor4	0.25	0.16	0.15	0.007	114
corridor5	0.77	0.31	0.31	0.009	270
magistrale1	2.19	0.20	0.24	0.18	918
magistrale2	3.11	0.46	0.27	0.30	561

monocular-inertial Rover-SLAM are presented in Table II. The trajectory visualization on the corridor3 sequence is shown in Fig. 6. Rover-SLAM achieved the lowest accumulated drift in most sequences and demonstrated superior performance. This result reflects the stability and reliability of our method.

2) *Stereo-Inertial/Stereo Tests*: For the stereo-inertial sensor setups, several representative stereo-inertial SLAM systems, including VINS-Fusion [36], BASALT [37], Kimera [38], and ORB-SLAM3 [16], are chosen as contrast targets. The test

results for stereo-inertial Rover-SLAM are presented on the left of Table III. It can be observed that Rover-SLAM achieved the best performance in 8 out of the 11 sequences. For the stereo sensor setups, we selected several representative stereo SLAM systems, including VINS-Fusion [36], SVO [39], and ORB-SLAM2 [40]. The test results for stereo Rover-SLAM are presented in the right of Table III. It can be observed that Rover-SLAM achieved the best performance in 6 out of 10 sequences. The superior performance of our method across multiple sensor configurations underscores its robustness and high scalability.

3) *Monocular Tests*: For the monocular sensor mode, we selected direct vSLAM DSO [13], semi-direct vSLAM SVO [39], sparse feature SLAM ORB-SLAM3, and dense feature SLAM DSM [41] for comparison, all of which are representative visual SLAM systems. The ATE of the estimated discrete poses is shown in Table IV. Among the 11 sequences in EuRoC, Rover-SLAM achieved SOTA results in 9 of them. In sequence V203, characterized by significant motion blur and photometric changes, ORB-SLAM3's monocular mode failed to track features. However, due to Rover-SLAM's robust feature tracking and loop closure detection ability, it quickly relocalized or merged multiple loops, despite occasional tracking failures, minimizing their impact.

D. System Functional Visualization Verification

1) *Adaptive Feature Extraction Verification*: The qualitative analysis of the adaptive confidence threshold is illustrated in Fig. 7, showing that when the scene is simple and there are many feature points, some feature points can be filtered out to reduce computational overhead. In contrast, when the scene is complex, lowering the threshold allows for the extraction of more feature points, ensuring stable tracking.

2) *Tracking Based on Learnable Feature Matching Verification*: We extracted superpoint feature points from two images, matching them using both learning-based methods and geometric methods. The results are shown in Fig. 8, which illustrates that the learning-based feature matching network ensures accurate and robust matching relationships.

We integrated the learning-based methods into different aspects of the tracking thread, as shown in Fig. 9. It can be seen that the learning-based methods improve the matching performance at these stages. The learning-based matching

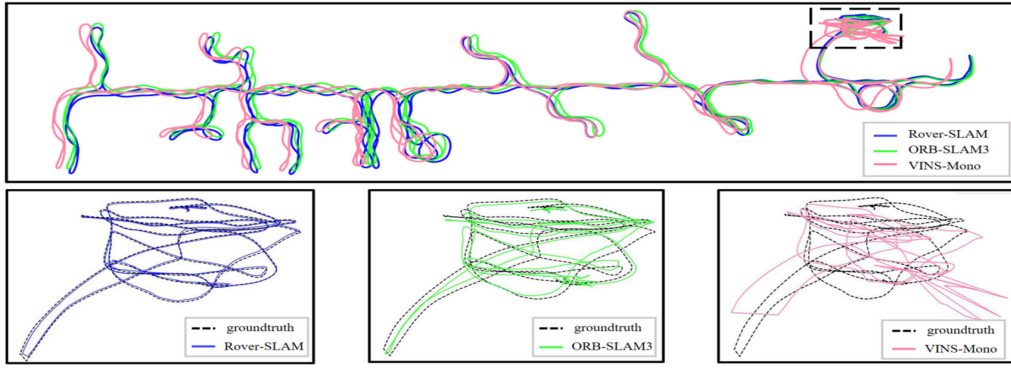


Fig. 6. Representative trajectory visualization of Rover-SLAM on the TUM-VI dataset. In this test, only the last room contains the ground truth (within the dashed box), and details of the trajectory (bottom).

TABLE III
RMSE[M] OF ATE WITH SOTA STEREO-VI/STEREO METHODS ON EuRoC

Dataset	Stereo-inertial					Stereo			
	VINS-Fusion	BASALT	Kimera	ORB-SLAM3	Ours	VINS-Fusion	SVO	ORB-SLAM2	Ours
MH01	0.166	0.080	0.080	0.036	0.024	0.540	0.040	0.035	0.033
MH02	0.152	0.060	0.090	0.033	0.023	0.460	0.070	0.018	0.025
MH03	0.125	0.050	0.110	0.035	0.019	0.330	0.270	0.028	0.026
MH04	0.280	0.100	0.150	0.051	0.047	0.780	0.170	0.119	0.093
MH05	0.284	0.080	0.240	0.082	0.051	0.500	0.120	0.060	0.057
V101	0.076	0.040	0.050	0.038	0.033	0.550	0.040	0.035	0.041
V102	0.069	0.020	0.110	0.014	0.018	0.230	0.040	0.020	0.025
V103	0.114	0.030	0.120	0.024	0.022	×	0.070	0.048	0.063
V201	0.066	0.030	0.070	0.032	0.046	0.230	0.050	0.037	0.041
V202	0.091	0.020	0.100	0.014	0.012	0.200	0.090	0.035	0.032
V203	0.096	×	0.190	0.024	0.032	×	0.790	×	0.558

TABLE IV
RMSE[M] OF ATE WITH SOTA MONOCULAR METHODS ON EuRoC

Dataset	DSO	SVO	DSM	ORB-SLAM3	Ours
MH01	0.046	0.100	0.039	0.016	0.007
MH02	0.046	0.120	0.036	0.027	0.032
MH03	0.172	0.410	0.055	0.028	0.022
MH04	3.810	0.430	0.057	0.138	0.014
MH05	0.110	0.300	0.067	0.072	0.038
V101	0.089	0.070	0.095	0.033	0.021
V102	0.107	0.210	0.059	0.015	0.015
V103	0.903	×	0.076	0.033	0.037
V201	0.044	0.110	0.056	0.023	0.021
V202	0.132	0.110	0.057	0.029	0.016
V203	1.152	1.080	0.784	×	0.031

outperforms the projection and BoW matching methods in ORB-SLAM3, resulting in a more uniform and stable distribution of matched map points.

3) *Local Mapping Based on Learnable Feature Matching Verification*: The impact of triangulating map points is depicted in Fig. 10. It can be observed that compared to ORB-SLAM3, the map points constructed by Rover-SLAM exhibit superior structural information of the scene. Additionally, they are distributed more evenly and extensively.

4) *Deep Learning Modules Replacement Verification*: We integrated XFeat [31] into the framework as a replacement for SuperPoint, denoted as ORB-SLAM3 (XFeat). Additionally, we replaced LightGlue with SuperGlue [32] for feature matching, denoted as Rover-SLAM (SuperGlue). Since XFeat



Fig. 7. Adaptive confidence threshold visualization of Rover-SLAM. The subplots in the first row show the extracted feature points without setting an adaptive threshold, while those in the second row correspond to the extracted feature points after setting the adaptive threshold. It can be observed that after applying the adaptive threshold, the number of extracted feature points can be adjusted according to the environment.

still employs geometry-based matching methods, it exhibited unstable tracking in complex scenes, as shown in the left part of Fig. 11. It can be observed that in normal scenes, Rover-SLAM is able to reconstruct and track more map points, such as the keyboard portion in the figure. In shaky scenes, Rover-SLAM continues to track effectively, whereas ORB-SLAM3 (XFeat) struggles to track map points, leading to tracking failures. The visualization of the estimated trajectory on the EuRoC V201 dataset is shown in the right part of

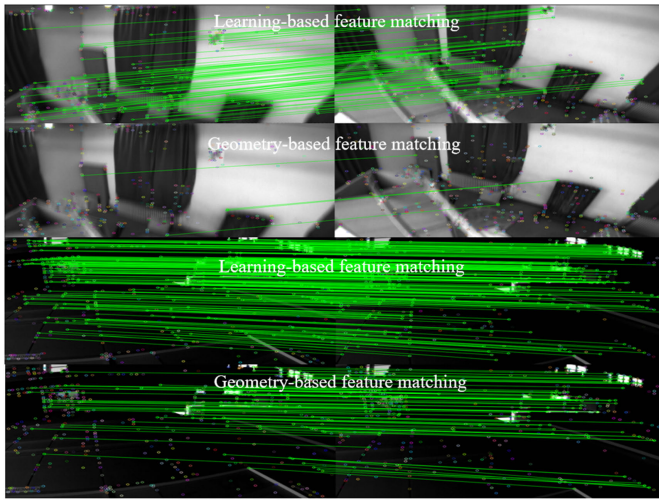


Fig. 8. Superpoint feature matching comparison between learning-based method and Geometry-based method. The first and third rows show the matching results of learning-based methods, while the second and fourth rows show the matching results of traditional geometry-based methods. It can be observed that in challenging scenarios, such as motion blur and low-light conditions, learning-based methods are able to match more comprehensive feature points.

TABLE V

VERIFICATION OF DEEP LEARNING MODULE REPLACEMENTS: ATE, RUNNING TIMES, AND GPU MEMORY USAGE ON THE EUROC V201 SEQUENCE

method	ATE	running times	GPU Memory
Rover-SLAM	0.024	32.6ms	2271MB
ORB-SLAM3(XFeat)	0.053	21.9ms	1091MB
Rover-SLAM(SuperGlue)	0.027	47.2ms	3128MB

Fig. 11. It can be seen that ORB-SLAM3 (XFeat) exhibits a larger deviation from the ground-truth trajectory, whereas Rover-SLAM and Rover-SLAM (SuperGlue) are closer to the ground truth. Moreover, we conducted a quantitative analysis, with results shown in Table V. Experimental results indicate that Rover-SLAM (SuperGlue) achieved localization accuracy comparable to the original Rover-SLAM. However, due to the lack of an early exit mechanism in LightGlue, it resulted in a significant increase in running times and GPU memory. The performance of these methods does not surpass that of Rover-SLAM. Although ORB-SLAM3 (XFeat) has an advantage in runtime and GPU memory consumption, its localization accuracy is significantly inferior to that of Rover-SLAM. These results also demonstrate that our framework exhibits good transferability, albeit requiring adaptive improvements to specific deep learning algorithms.

E. Tracking Robustness Evaluation

Since the self-collected dataset simulates various challenging scenarios, we use it to evaluate the robustness of our method and compare it with ORB-SLAM3. As shown in Fig. 12, our method effectively extracts and tracks map points in low-texture areas and dark regions. The estimated trajectory, displayed in Fig. 4 (bottom left), remains stable despite significant illumination changes. In contrast, ORB-SLAM3 loses

TABLE VI
ABLATION STUDY. ATE AND RELATIVE POSE
ERROR ON EUROC V201 SEQUENCE

method	ATE(RMSE)	ATE(std)	GPU Memory(MB)
(1):ORB-SLAM3	0.0760	0.0419	×
(2):1+SP	0.0518	0.0235	1829
(3):2+LT	0.0391	0.0197	2172
(4):3+AT	0.0384	0.0179	1921
(5): 4+LL	0.0249	0.0060	2190

track of map points and exhibits increasing drift or even loses tracking altogether under such conditions. Experimental results demonstrate that our method demonstrates strong robustness in these challenging environments, exhibiting excellent adaptability and stability.

F. Time Costs Evaluation

The SLAM system's tracking and local mapping threads consume most of the system resources, and the detailed runtime analysis of these two threads is shown in Fig. 13. Owing to the acceleration effect of deploying deep learning models with ONNX Runtime, it is evident that in the tracking thread, feature extraction for each frame takes approximately 7 ms, pose estimation takes around 10 ms, and the total tracking time is around 30 ms, allowing real-time tracking. Additionally, because the algorithm reconstructs a large number of map points, the map point creation module in the local mapping thread requires a longer time, about 100 ms, and the total thread time is 180 ms. However, since this thread does not need to update every frame, its impact on the overall system's real-time performance remains limited. Experimental results show that Rover-SLAM can track at a specific frame rate of 30 frames/s, demonstrating good real-time performance. Additionally, Rover-SLAM maintains a GPU memory consumption of around 2300 MB during runtime, which can be accommodated by most GPUs available at present. However, reducing memory consumption and increasing runtime speed for easier deployment on embedded devices remains an area for further research.

G. Ablation Study

In this section, a series of tests was conducted to evaluate the efficacy of the modules proposed in our study. We chose to use ORB-SLAM3 as the baseline for the ablation study, denoted as method 1. First, we added the learning-based feature extraction module (SP) to the baseline, represented as method 2. Building on method 2, we replaced the original tracking module with a learning-based tracking module (LT), denoted as method 3. Next, we incorporated the proposed adaptive confidence threshold into method 3, resulting in method 4. Finally, we replaced the local mapping component in method 4 with a learning-based local mapping module, denoted as method 5. By incrementally adding the aforementioned modules, method 5 can also be considered as Rover-SLAM. The experimental results are presented in Table VI.

It can be observed that by replacing the original ORB features with learning-based features, we achieved improve-

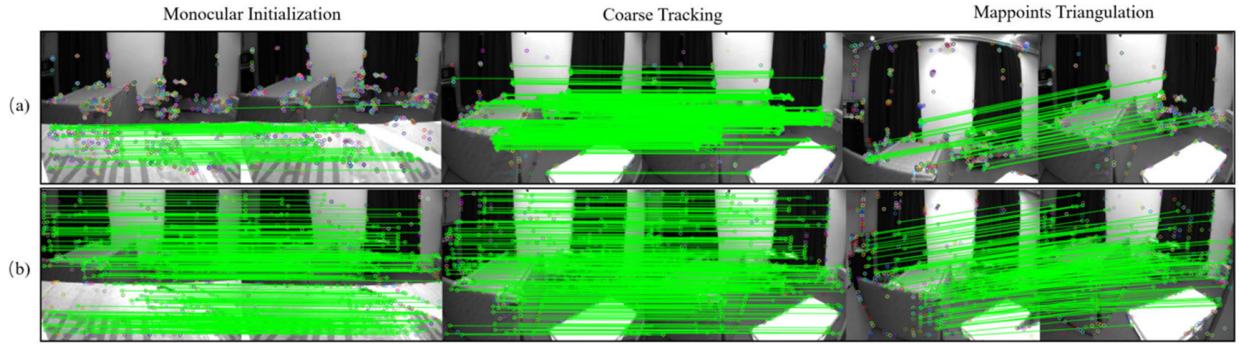


Fig. 9. Comparison of map point matching performance at different stages. (a) and (b) Matching results of ORB-SLAM3 and Rover-SLAM, respectively. Each column represents a different stage of the system: monocular initialization, coarse tracking, and map point triangulation. In the baseline system, coarse tracking employs a projection-based matching method, while monocular initialization and map point triangulation utilize DBoW-based matching methods.

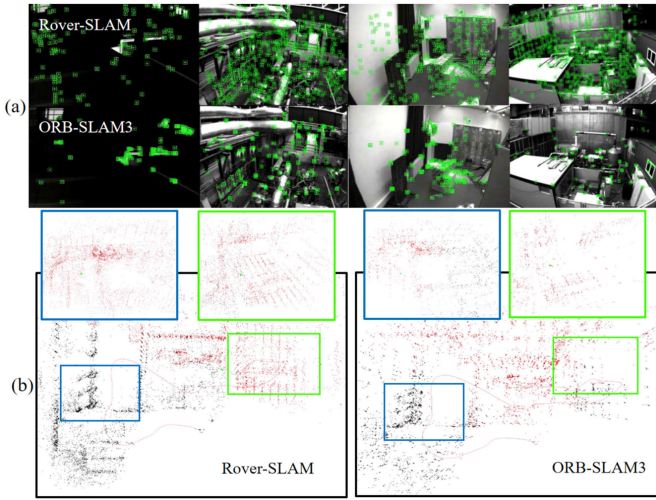


Fig. 10. Map point comparison between Rover-SLAM and ORB-SLAM3. (a) Comparison of tracked map points, with the first row displaying the tracking results of Rover-SLAM and the second row showing the tracking results of ORB-SLAM3. (b) Comparison of the reconstructed sparse point clouds, with rectangles of different colors highlighting the corresponding regions with significant differences.

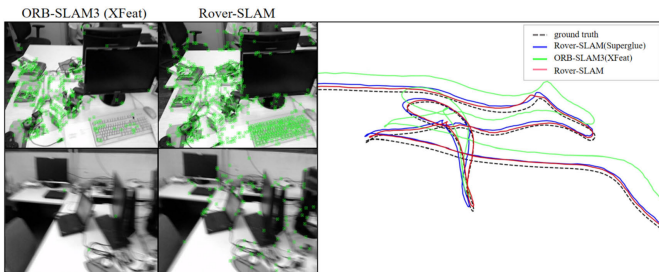


Fig. 11. Deep learning modules replacement visualization. Tracking performance comparison between Rover-SLAM and ORB-SLAM3 (XFeat) on the TUM-VI dataset, where the first column corresponds to ORB-SLAM3 (XFeat) and the second column to Rover-SLAM (left).: Partial trajectory visualization on the EuRoC V201 dataset (right).

ment in the ATE metric. Additionally, replacing the tracking and local mapping components with learning-based methods also improved localization accuracy. Furthermore, by introducing the adaptive confidence threshold mechanism in the feature extraction phase, we found that it reduced mem-

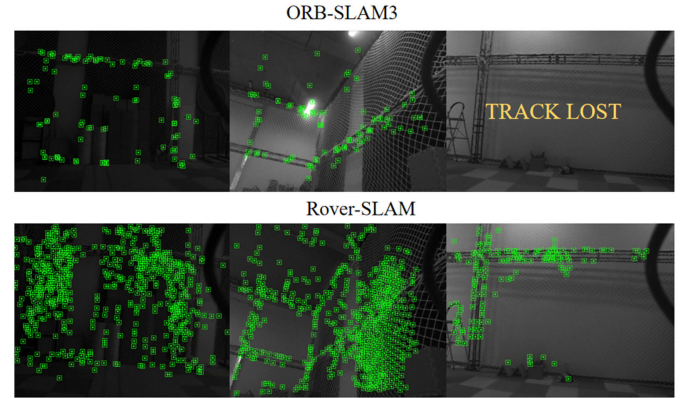


Fig. 12. Tracking performance comparison of Rover-SLAM and ORB-SLAM3 on self-collected dataset.

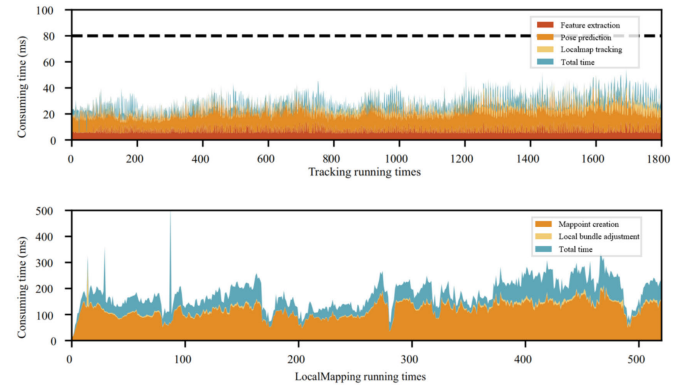


Fig. 13. Time costs visualization of Rover-SLAM. The vertical axis represents the time consumed at each stage, while the horizontal axis represents the number of complete runs of the thread.

ory consumption while achieving a slight improvement in accuracy. These experiments validate the effectiveness of the proposed modules.

V. CONCLUSION

This article introduces Rover-SLAM, a real-time, robust, and versatile visual-SLAM framework that integrates SOTA learning networks to improve the performance of traditional vSLAM solutions in challenging scenarios. In Rover-SLAM,

the same features and matching networks are utilized for all tasks, i.e., tracking, mapping, relocalization, and loop closing. Additionally, novel schemes such as adaptive feature filtering, learning-based local map tracking, deep feature BoW, and versatile usage expansion are implemented to enhance efficiency, simplicity, reliability, and user-friendliness. Both quantitative and qualitative analyses confirm that Rover-SLAM signifies a significant advancement in the vSLAM domain, delivering robust performance in complex environments while maintaining efficiency and scalability.

However, our approach also has certain limitations. First, it has high hardware requirements, relying on high-performance GPUs. Second, compared to mainstream traditional SLAM systems, the tracking speed of our method still requires further improvement. Furthermore, the stability of localization accuracy needs enhancement. Therefore, our future research will focus on simplifying deep learning models, improving tracking speed, and enabling deployment on more resource-constrained hardware. Additionally, we will refine optimization algorithms to reduce the randomness of localization accuracy and ensure more consistent performance.

Revisiting the question “How to use deep-learning network in SLAM,” we believe the answer depends on the application, but quite often the answer is *hybrid*. The following reasons are specified.

- 1) *Real-Time Performance*: Deep learning networks designed for specific tasks rather than the complete SLAM task often do not require overly complex model structures, allowing for lightweight implementations. Leveraging GPU ensures the real-time performance of SLAM systems. It can be observed in the experiment that the feature extraction module in Rover-SLAM only requires 6 ms to process a new incoming frame, and the system can perform real-time tracking at a speed of 30 frames/s. Therefore, hybrid SLAM exhibits high real-time performance.
- 2) *Interpretability*: SLAM is a complex system that requires coordinated cooperation among multiple modules. Instead of integrating a large-scale model for the entire SLAM procedure, integrating various learning models across these modules separately helps in troubleshooting problems when the system malfunctions. As shown in the system functional visualization verification, each module's function in Rover-SLAM can be tested and visualized independently, indicating that hybrid SLAM has high interpretability.
- 3) *Generalization*: End-to-end SLAM methods require abundant training samples and significant resource consumption to adjust network structures and retrain. However, the generalization capability of these methods remains limited in different environments, complex scenarios, and varying data inputs. Extensive experiments have demonstrated that Rover-SLAM achieves excellent results in various complex scenes and multiple sensor setups using only pretrained models, significantly reducing resource consumption. This evidence supports that hybrid SLAM has good generalization capabilities.

REFERENCES

- [1] J. Cheng, L. Zhang, Q. Chen, X. Hu, and J. Cai, “A review of visual SLAM methods for autonomous driving vehicles,” *Eng. Appl. Artif. Intell.*, vol. 114, Sep. 2022, Art. no. 104992.
- [2] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [3] C. Kerl, J. Sturm, and D. Cremers, “Dense visual SLAM for RGB-D cameras,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Nov. 2013, pp. 2100–2106.
- [4] A. I. Mourikis and S. I. Roumeliotis, “A multi-state constraint Kalman filter for vision-aided inertial navigation,” in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2007, pp. 3565–3572.
- [5] C. Cadena et al., “Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age,” *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.
- [6] Z. Zhu et al., “NICE-SLAM: Neural implicit scalable encoding for SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 12786–12796.
- [7] X. Peng, Z. Liu, W. Li, P. Tan, S. Cho, and Q. Wang, “DVI-SLAM: A dual visual inertial SLAM network,” 2023, *arXiv:2309.13814*.
- [8] D. Li et al., “DXSLAM: A robust and efficient visual SLAM system with deep features,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 4958–4965.
- [9] Y. Wang, B. Xu, W. Fan, and C. Xiang, “A robust and efficient loop closure detection approach for hybrid Ground/Aerial vehicles,” *Drones*, vol. 7, no. 2, p. 135, Feb. 2023, doi: [10.3390/drones7020135](https://doi.org/10.3390/drones7020135).
- [10] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-supervised interest point detection and description,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 224–236.
- [11] H. Chen et al., “ASpanFormer: Detector-free image matching with adaptive span transformer,” in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2022, pp. 20–36.
- [12] P. Lindenberger, P. E. Sarlin, and M. Pollefeys, “LightGlue: Local feature matching at light speed,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 17627–17638.
- [13] J. Engel, V. Koltun, and D. Cremers, “Direct sparse odometry,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [14] T. Qin, P. Li, and S. Shen, “VINS-mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [15] L. Xia, D. Meng, J. Zhang, D. Zhang, and Z. Hu, “Visual-inertial simultaneous localization and mapping: Dynamically fused point-line feature extraction and engineered robotic applications,” *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–11, 2022.
- [16] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM,” *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [17] X. Liu, S. Wen, J. Zhao, T. Z. Qiu, and H. Zhang, “Edge-assisted multi-robot visual-inertial SLAM with efficient communication,” *IEEE Trans. Automat. Sci. Eng.*, doi: [10.1109/TASE.2024.3376427](https://doi.org/10.1109/TASE.2024.3376427).
- [18] N. Yang, L. von Stumberg, R. Wang, and D. Cremers, “D3 VO: Deep depth, deep pose and deep uncertainty for monocular visual odometry,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1281–1292.
- [19] R. Li, S. Wang, Z. Long, and D. Gu, “UnDeepVO: Monocular visual odometry through unsupervised deep learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 7286–7291.
- [20] Z. Teed and J. Deng, “DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 16558–16569.
- [21] X. Wu, Z. Liu, Y. Tian, Z. Liu, and W. Chen, “KN-SLAM: Keypoints and neural implicit encoding SLAM,” *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–12, 2024.
- [22] Z. Teed, L. Lipson, and J. Deng, “Deep patch visual odometry,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–19.
- [23] J. Tang, L. Ericson, J. Folkesson, and P. Jensfelt, “GCNv2: Efficient correspondence prediction for real-time SLAM,” *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3505–3512, Oct. 2019.

- [24] H. M. S. Bruno and E. L. Colombari, "LIFT-SLAM: A deep-learning feature-based monocular visual SLAM method," *Neurocomputing*, vol. 455, pp. 97–110, Sep. 2021.
- [25] S. Wen, S. Tao, X. Liu, A. Babiarz, and F. Richard, "CD-SLAM: A real-time stereo visual-inertial SLAM for complex dynamic environments with semantic and geometric information," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 2517808.
- [26] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [27] C. Zhang, L. Chen, and S. Yuan, "ST-VIO: Visual-inertial odometry combined with image segmentation and tracking," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 10, pp. 8562–8570, Oct. 2020.
- [28] B. Fu and X. Yao, "Visual SLAM system integrating feature extraction based on vision transformer and cloud-edge technology," in *Proc. Int. Conf. Intell. Comput. Robot. (ICICR)*, Apr. 2024, pp. 73–77.
- [29] K. Xu, Y. Hao, S. Yuan, C. Wang, and L. Xie, "AirVO: An illumination-robust point-line visual odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2023, pp. 3429–3436.
- [30] G. Klein and D. Murray, "Parallel tracking and mapping on a camera phone," in *Proc. 8th IEEE Int. Symp. Mixed Augmented Reality*, Oct. 2009, pp. 83–86.
- [31] G. Potje, F. Cadar, A. Araujo, R. Martins, and E. R. Nascimento, "XFeat: Accelerated features for lightweight image matching," 2024, *arXiv:2404.19174*.
- [32] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperGlue: Learning feature matching with graph neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4938–4947.
- [33] Y. Wang, B. Xu, W. Fan, and C. Xiang, "A robust and efficient loop closure detection approach for hybrid ground/aerial vehicles," *Drones*, vol. 7, no. 2, p. 135, Feb. 2023.
- [34] M. Burri et al., "The EuRoC micro aerial vehicle datasets," *Int. J. Robot. Res.*, vol. 35, no. 10, pp. 1157–1163, Sep. 2016.
- [35] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stückler, and D. Cremers, "The TUM VI benchmark for evaluating visual-inertial odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1680–1687.
- [36] T. Qin, J. Pan, S. Cao, and S. Shen, "A general optimization-based framework for local odometry estimation with multiple sensors," 2019, *arXiv:1901.03638*.
- [37] V. Usenko, N. Demmel, D. Schubert, J. Stückler, and D. Cremers, "Visual-inertial mapping with non-linear factor recovery," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 422–429, Apr. 2020.
- [38] Y. Tian, Y. Chang, F. Herrera Arias, C. Nieto-Granda, J. P. How, and L. Carlone, "Kimera-multi: Robust, distributed, dense metric-semantic SLAM for multi-robot systems," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2022–2038, Aug. 2022.
- [39] C. Forster, M. Pizzoli, and D. Scaramuzza, "SVO: Fast semi-direct monocular visual odometry," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 15–22.
- [40] R. Mur-Artal and J. D. Tardós, "Visual-inertial monocular SLAM with map reuse," *IEEE Robot. Automat. Lett.*, vol. 2, no. 2, pp. 796–803, Apr. 2017.
- [41] J. Zubizarreta, I. Aguinaga, and J. M. M. Montiel, "Direct sparse mapping," *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1363–1370, Aug. 2020.